

Cross-System Data Drift & Trust Monitoring Platform

Tech Stack: Python | PySpark | SQL | Databricks | Delta Lake

1. Abstract

Modern enterprises operate across multiple interconnected systems such as CRM, Billing, and Analytics platforms. Data flowing through these systems is often prone to inconsistency, silent drift, and mismatches that go undetected until they cause significant business impact.

This project proposes a Cross-System Data Drift & Trust Monitoring Platform — a centralized solution aimed at detecting, reporting, and scoring data discrepancies across enterprise systems in near real time. The platform will be built using PySpark, SQL, Databricks, and Delta Lake, providing automated alerts, drift detection, and a data trust scoring framework to improve overall data reliability.

2. Problem Statement

Enterprises rely on multiple systems such as CRM, Billing, and Analytics that often fail to stay synchronized due to independent update cycles and integration issues. This results in conflicting, missing, or duplicate records across systems, leading to inaccurate reporting and unreliable business insights.

Data pipelines responsible for moving and transforming data can silently introduce errors such as data loss, duplication, or incorrect calculations. These issues are difficult to detect without a centralized monitoring mechanism. In critical domains like banking and e-commerce, even small inconsistencies can cause financial losses, compliance risks, and poor customer experience.

Currently, there is no unified and automated solution to continuously compare and validate data across systems in near real time. Therefore, there is a need to build a scalable monitoring platform using Databricks, PySpark, SQL, and Delta Lake to detect data drift, identify discrepancies, and ensure data consistency across enterprise systems.

3. Proposed Solution

The proposed platform will provide:

- Centralized Data Monitoring across CRM, Billing, and Analytics systems
- Automated cross-system comparison to detect missing, extra, and mismatched records
- Drift detection using statistical and rule-based techniques
- Configurable alerting system for anomalies
- Data Trust Scoring framework to quantify dataset reliability

4. Objectives

- Ensure consistency of data across all systems
- Detect mismatches automatically at record and field level
- Enable near real-time monitoring (batch + streaming)
- Improve data reliability for business decisions

5. Proposed Architecture

The platform follows a Medallion (Lakehouse) Architecture:

Ingestion: Data from CRM, Billing, Analytics (batch + streaming) using Python and Spark Streaming

Bronze Layer: Raw data stored in Delta Lake

Silver Layer: Cleaned, standardized, and compared data using PySpark and SQL

Gold Layer: Aggregated and drift analysis datasets

Monitoring: Drift detection, trust scoring, and alerts via Databricks notebooks

6. Planned Data Sources

- CRM Data: Customer records, interactions
- Billing Data: Transactions, invoices
- Analytics Data: KPIs, reports
- Streaming Data: Real-time events

- Synthetic Data: 10,000+ generated records for testing

7. Implementation Approach

Data Processing:

- PySpark for transformations and joins
- SQL for aggregation and scoring

Comparison Logic:

- Missing Records Detection
- Extra Records Detection
- Aggregation Mismatch

Drift Detection:

- Volume Drift
- Distribution Drift
- Schema Drift

8. Expected Outcomes

- Early detection of data quality issues
- Improved reliability across systems
- Trust-scored datasets for decision making
- Reusable monitoring framework

9. Anticipated Challenges

- Handling large-scale data efficiently
- Schema mismatches across systems
- Complexity in streaming comparisons

10. Future Scope

- AI-based anomaly detection
- Automated data correction
- Multi-cloud deployment support

11. Conclusion

Data trust is critical for reliable analytics and business decisions. This platform provides a scalable and automated solution for detecting drift, comparing systems, and ensuring data consistency using a modern Databricks-based Lakehouse architecture.